

Logic Synthesis and Verification

Programming Assignment 1

Yung-Hsiang Yang B11901064 September 21, 2026

1 Exercise 2: two-bit unsigned multiplier

The file `mul.blif` implements $Y = AB$, with $A = (a_1a_0)$, $B = (b_1b_0)$, and $Y = (y_3y_2y_1y_0)$. Four partial products and two half-adder stages implement

$$\begin{aligned} p_{00} &= a_0b_0, & p_{01} &= a_0b_1, & p_{10} &= a_1b_0, & p_{11} &= a_1b_1, \\ y_0 &= p_{00}, & y_1 &= p_{01} \oplus p_{10}, & c_1 &= p_{01}p_{10}, \\ y_2 &= p_{11} \oplus c_1, & y_3 &= p_{11}c_1. \end{aligned}$$

Each AND and XOR is expressed as a BLIF `.names` table. The input order is `a1 a0 b1 b0`; the output order is `y3 y2 y1 y0`.

An independent reference BLIF was generated from integer multiplication, rather than the half-adder equations. ABC's `cec` reported **Networks are equivalent**. The reference covers all sixteen input pairs:

$A \setminus B$	0	1	2	3
0	0	0	0	0
1	0	1	2	3
2	0	2	4	6
3	0	3	6	9

In particular, $3 \cdot 3 = 9 = (1001)_2$.

Experimental setup and measured statistics

The experiments use the course ABC checkout, built with `make -j4` and bundled CUDD on an Apple Silicon Mac. Each comparison starts by reading the same multiplier. Screenshots show actual ABC command output and ABC-generated diagrams displayed together for readability. Graphviz renders the diagrams; terminal color-control sequences are omitted.

Representation	Logic nodes	Edges	Levels	Function statistic
Original SOP	8	16	3	<code>cube</code> = 10
Local AIG (<code>aig</code>)	8	16	3	<code>aig</code> = 12
Strashed AIG	—	—	4	<code>and</code> = 12
Local BDD (<code>bdd</code>)	8	16	3	<code>bdd</code> = 16
Collapsed BDD	4	14	1	<code>bdd</code> = 14
SOP after <code>strash; logic</code>	12	24	4	<code>cube</code> = 12

The function counters are ABC's reported values, not interchangeable measures of network size. Every case has four inputs, four outputs, and no latches.

Reading and displaying the original network

```
read mul.blif
print_stats
show
```

The original network has eight internal nodes: four partial-product gates, two XOR gates, and two carry gates. XOR nodes have two cubes, so the total is ten cubes. Its longest path has three logic levels.

Exercise 2: original SOP network

ABC command output and generated diagram

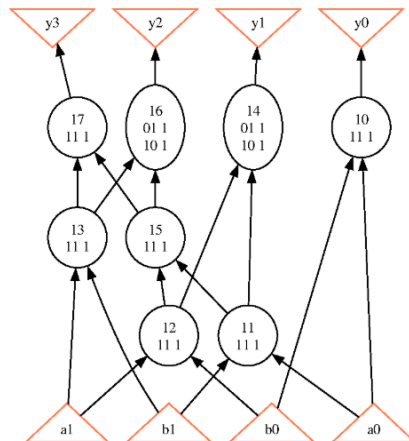
```
ABC command line: "source setup.rc; read mul.blif; print_stats; show".
```

```
Loading resource file "/Users/ooyang/Downloads/2026/LSV/LSV-PA/abc.rc".
```

```
mul          : i/o =   4/   4  lat =   0  nd =   8  edge =   16  cube =   10  lev =   3
```

Network structure visualized by ABC
Benchmark "mul". Time was Mon Sep 21 15:58:10 2026.

The network contains 8 logic nodes and 0 latches.



Original SOP network and its measured statistics.

Structural hashing

```
read mul.blif
strash
print_stats
show
```

strash constructs a global AIG with two-input AND nodes and complemented edges. Each XOR requires a small AND/inverter implementation. The resulting network has twelve AND nodes and four levels. More AIG nodes than original logic nodes is not a correctness problem: the node types represent different amounts of logic.

Exercise 2: structurally hashed AIG

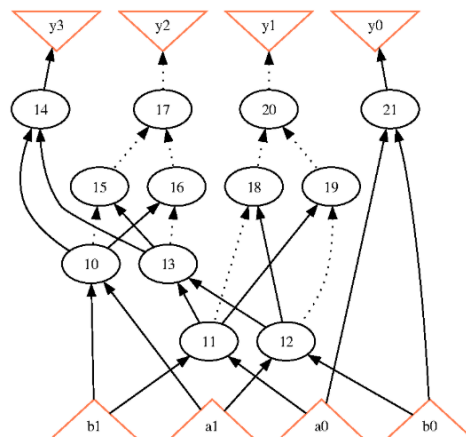
ABC command output and generated diagram

```
ABC command line: "source setup.rc; read mul.blif; strash; print_stats; show".

Loading resource file "/Users/ooyang/Downloads/2026/LSV/LSV-PA/abc.rc".
mul          : i/o =   4/   4 lat =   0 and =  12 lev =   4
```

Network structure visualized by ABC
Benchmark "mul". Time was Mon Sep 21 15:58:11 2026.

The network contains 12 logic nodes and 0 latches.



Structurally hashed AIG. Inversion is represented on edges.

Collapsing and displaying all output BDDs

```
read mul.blif
strash
collapse
print_stats
show_bdd -g
```

After collapse, there are four logic nodes, one per output, and one logic level. Their functions are expressed directly in terms of primary inputs. The network has fourteen fanin edges, and ABC reports `bdd = 14`. The `-g` option displays all output functions; without it, `show_bdd` defaults to one output's driver.

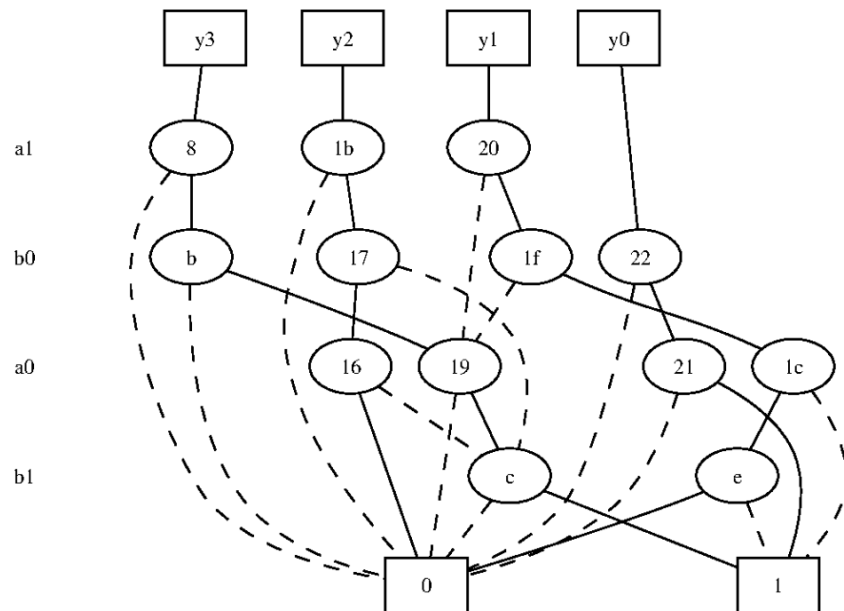
Exercise 2: collapsed global BDDs

ABC command output and generated diagram

```
ABC command line: "source setup.rc; read mul.blif; strash; collapse; print_stats; show_bdd -g".
```

```
Loading resource file "/Users/ooyang/Downloads/2026/LSV/LSV-PA/abc.rc".
```

```
mul          : i/o =   4/   4  lat =   0  nd =   4  edge =   14  bdd =   14  lev = 1
```



Global view of all four multiplier output functions.

2 Exercise 3: representation comparisons

2.1 Local AIG versus structurally hashed AIG

```
read mul.blif
aig
print_stats
show
```

aig preserves the eight-node logic network and replaces each node's local function representation by an AIG. Its fanins are still the signals feeding that logic node. In contrast, **strash** constructs one global AND/inverter network whose leaves are the circuit inputs.

For this multiplier, **aig** reports twelve ANDs in the local function representation, while **strash** also produces twelve AND nodes. There is no observed reduction in that AND count. The structural difference is visible in the topology: eight logic nodes and three levels versus twelve global AND nodes and four levels. Structural hashing merges structurally identical nodes; it is not arbitrary Boolean minimization. Local AIGs should not be described simply as “unhashed AIGs.”

Exercise 3: local AIG representation

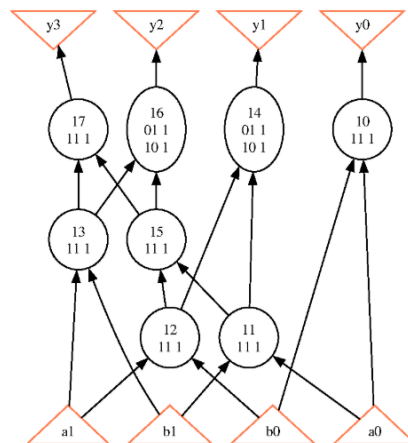
ABC command output and generated diagram

```
ABC command line: "source setup.rc; read mul.blif; aig; print_stats; show".

Loading resource file "/Users/ooyang/Downloads/2026/LSV/LSV-PA/abc.rc".
mul          : i/o =   4/   4 lat =   0 nd =   8 edge =   16 aig =   12 lev = 3
```

Network structure visualized by ABC
Benchmark "mul". Time was Mon Sep 21 15:58:13 2026.

The network contains 8 logic nodes and 0 latches.



Local AIG representation; compare with the global AIG on the previous pages.

2.2 Local BDD versus collapsed BDD: network structure

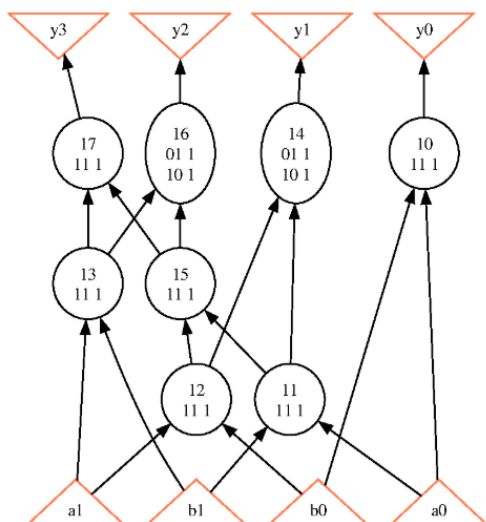
```
read mul.blif; bdd; print_stats; show
read mul.blif; collapse; print_stats; show
```

`bdd` changes local node functions without flattening the network: eight nodes, sixteen edges, and three levels remain. Its `bdd` counter is sixteen. `collapse` composes the functions into output functions over primary inputs, leaving four nodes, fourteen edges, and one level, with a reported BDD counter of fourteen.

These observations distinguish the circuit's structure from the size of its function representations. Collapse reduces these measurements for this example; it need not make BDDs small for an arbitrary circuit or ordering. The diagram regions below are cropped from the full screenshots to keep the node labels readable.

Network structure visualized by ABC
Benchmark "mul". Time was Mon Sep 21 15:58:14 2026.

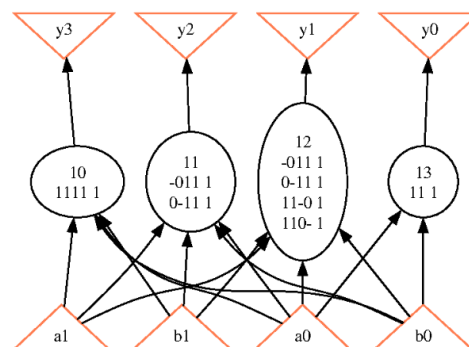
The network contains 8 logic nodes and 0 latches.



Local BDD network: internal signals remain as fanins.

Network structure visualized by ABC
Benchmark "mul". Time was Mon Sep 21 15:58:16 2026.

The network contains 4 logic nodes and 0 latches.

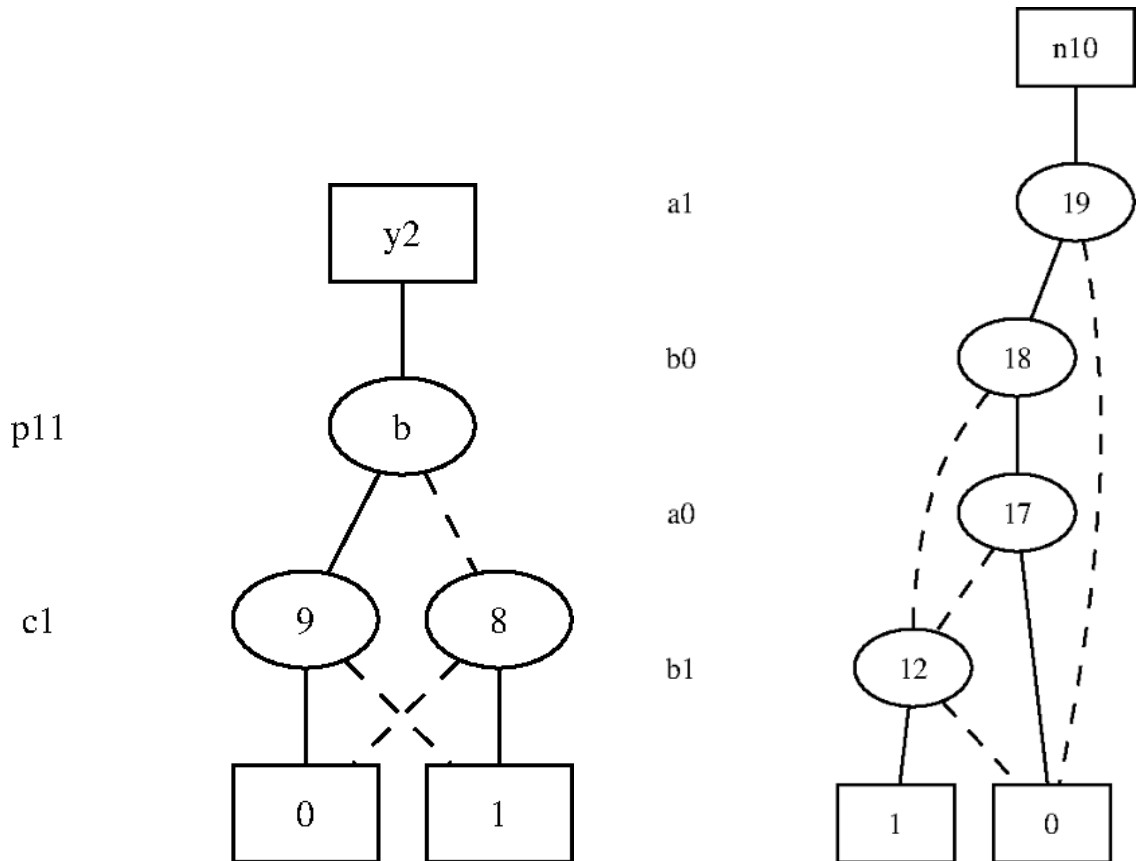


Collapsed network: each output function depends directly on primary inputs.

Local and global functions for y_2

```
read mul.blif; bdd; show_bdd y2
read mul.blif; collapse; show_bdd y2
```

Before collapse, $y_2 = p_{11} \oplus c_1$ is a local two-variable function. After collapse, the intermediate signals are substituted by their definitions, so the BDD tests primary inputs instead. Using `show_bdd -g` for both cases would instead request a global view in both cases and would not expose this local-versus-global distinction.



Local BDD for y_2 , in terms of its two immediate fanins.

Collapsed BDD for y_2 , in terms of primary inputs.

2.3 Converting a strashed AIG to SOP logic

```
read mul.blif
strash
logic
print_stats
lsv_print_nodes
```

`logic` converts the strashed AIG into a logic network with SOP node functions. It is the necessary representation conversion; applying `sop` directly to a strashed network is not sufficient because `sop` requires a logic network.

The resulting network has twelve nodes, twenty-four edges, twelve cubes, and four levels. The `lsv_print_nodes` listing confirms SOP data, including 11 1 for AND nodes and complemented-input cubes such as 01 1. Some functions use an OFF-set cube such as 00 0; a zero-valued cube is valid and should not be mistaken for a missing function. Conversion preserves the represented function, but does not reconstruct the original eight-node XOR-level design.

Exercise 3: strashed AIG to SOP logic

ABC command output (beginning of the node listing shown below)

```
ABC command line: "source setup.rc; read mul.blif; strash; logic; print_stats; lsv_print_nodes".

Loading resource file "/Users/ooyang/Downloads/2026/LSV/LSV-PA/abc.rc".
mul          : i/o =   4/   4 lat =   0 nd =   12 edge =   24 cube =   12 lev =  4
Object Id = 9, name = n9
  Fanin-0: Id = 1, name = a1
  Fanin-1: Id = 3, name = b1
The SOP of this node:
11 1
Object Id = 10, name = n10
  Fanin-0: Id = 2, name = a0
  Fanin-1: Id = 3, name = b1
The SOP of this node:
11 1
Object Id = 11, name = n11
  Fanin-0: Id = 1, name = a1
  Fanin-1: Id = 4, name = b0
The SOP of this node:
11 1
Object Id = 12, name = n12
  Fanin-0: Id = 10, name = n10
  Fanin-1: Id = 11, name = n11
The SOP of this node:
11 1
Object Id = 13, name = n13
  Fanin-0: Id = 9, name = n9
  Fanin-1: Id = 12, name = n12
The SOP of this node:
11 1
Object Id = 14, name = n14
  Fanin-0: Id = 9, name = n9
  Fanin-1: Id = 12, name = n12
```

Beginning of the actual SOP node listing after conversion.